Informe: Persistencia de Datos, Modelado de Estructuras y Metodología 5R en C+++

Luca Di Guglielmo 5°3°

E.E.S.T N°1 de Vicente López
Laboratorio de Programación 5°3°.
Profesor: Mansilla Munoz York Elias & Yamil Ganduglia
Fecha: 21/10/2026

Introducción

El presente informe técnico se enfoca de manera exclusiva en el análisis del código fuente desarrollado en lenguaje C++ para la aplicación interactiva de Piedra, Papel o Tijera. La solución implementada aborda la problemática fundamental de la volatilidad de la memoria RAM mediante el diseño de estructuras de datos personalizadas y el manejo de flujos de archivos no volátiles. A lo largo de este documento, se desglosan los componentes centrales del programa: la definición de registros con struct, la persistencia de partidas en disco mediante ofstream, la lectura y reconstrucción del historial con ifstream, la gestión de búferes de entrada y la generación aleatoria de jugadas por parte de la Inteligencia Artificial.

Desarrollo

Modelado de Datos con struct Partida

Para organizar la información de cada enfrentamiento de forma coherente, el código define una estructura personalizada denominada Partida. Esta entidad encapsula variables de diferentes tipos que describen el estado completo de un juego:

```
// 1. Estructura de datos para representar una partida
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};
```

Al agrupar estos cinco atributos bajo un solo tipo de dato, el programa simplifica la firma de las funciones. En lugar de transmitir múltiples argumentos individuales, la función de guardado recibe una referencia constante de la estructura (const Partida& p), reduciendo el sobrecosto de memoria por copia de datos.

Escritura Persistente en Disco mediante ofstream.

La persistencia en almacenamiento no volátil se logra mediante la librería de flujos de archivos <fstream>. Específicamente, la función encargada de registrar los resultados hace uso de la clase ofstream configurada en modo de adición (append):

```
// ios::app permite agregar al final sin borrar partidas anteriores
ofstream archivo("src/historial_partidas.txt", ios::app);

if (!archivo.is_open()) {
    cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
    return;
}

// Guardado de datos separados por comas (Formato CSV)
archivo << p.idPartida << ","
    << p.jugador << ","
    << p.eleccionJugador << ","
    << p.eleccionIA << ","
    << p.resultado << endl;

archivo.close();
cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." << endl;
```

El uso del flag ios::app resulta indispensable para situar el puntero de escritura al final del archivo existente, evitando la sobrescritura accidental de partidas previas. Cada registro se almacena en formato delimitado por comas (CSV), garantizando una estructura uniforme para su posterior procesamiento.

Lectura y Procesamiento del Historial mediante ifstream

Para consultar el historial almacenado sin recargar datos en memoria permanente, la aplicación emplea la clase ifstream. La lectura se ejecuta línea por línea mediante la función getline() utilizando la coma como carácter delimitador:

```
ifstream archivo("src/historial_partidas.txt");

if (!archivo.is_open()) {
    cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
    return;
}

cout << "\n=====================================================" << endl;
cout << " HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO     " << endl;
cout << "=====================================================" << endl;

string idStr, jugador, elJug, elIA, resultado;
int total = 0;

// Lectura secuencial delimitada por comas
while (getline(archivo, idStr, ',') &&
       getline(archivo, jugador, ',') &&
       getline(archivo, elJug, ',') &&
       getline(archivo, elIA, ',') &&
       getline(archivo, resultado)) {

    total++;
    cout << "Partida #" << idStr
         << " | Jugador: " << jugador
         << " | " << elJug << " vs " << elIA
         << " | Resultado: " << resultado << endl;
}

archivo.close();
```

Este esquema de evaluación condicional en el bucle while asegura que solo se procesen registros completos, previniendo errores de lectura en caso de líneas incompletas o archivos vacíos.

Control del Búfer de Entrada y Lógica de Selección Aleatoria

En la interacción por consola, la alternancia entre la lectura de datos numéricos mediante cin >> y cadenas completas con getline() puede provocar la permanencia del carácter

de salto de línea (\n) en el búfer de entrada. Para solucionar esta anomalía técnica, el código invoca cin.ignore() antes de solicitar el nombre del jugador, vaciando el búfer y previniendo la omisión inadvertida de lecturas.

Por otra parte, la Inteligencia Artificial simula la toma de decisiones inicializando la semilla del generador pseudoaleatorio mediante srand(time(0)) en el bloque principal. Posteriormente, determina su jugada evaluando la expresión (rand() % 3) + 1, la cual acota el resultado exactamente a tres valores discretos: 1 (Piedra), 2 (Papel) o 3 (Tijera).

Conclusión

El código en C + + analizado demuestra una implementación concisa y eficiente de los conceptos fundamentales de estructuración y persistencia de datos. Mediante el uso combinado de struct para el modelado en RAM y la librería <fstream> para la gestión no volátil en disco, la aplicación logra un equilibrio entre legibilidad, control de memoria y durabilidad de la información.

Referencias

Luca Di Guglielmo
Github: https://github.com/specIux/actividad-7-LPR
Gitlab: https://gitlab.com/specIux/actividad-7-lpr#

Laboratorio de Programación 5°3°
Actividad:
🎮📺🛠️ ACTIVIDAD 7: MINI JUEGO "PIEDRA, PAPEL O TIJERA" CON PERSI…